

City University of Hong Kong 2025-2026
Semester A CS3343 Software Engineering
Practice

Test Report
Group 19
Classic UNO Card Game

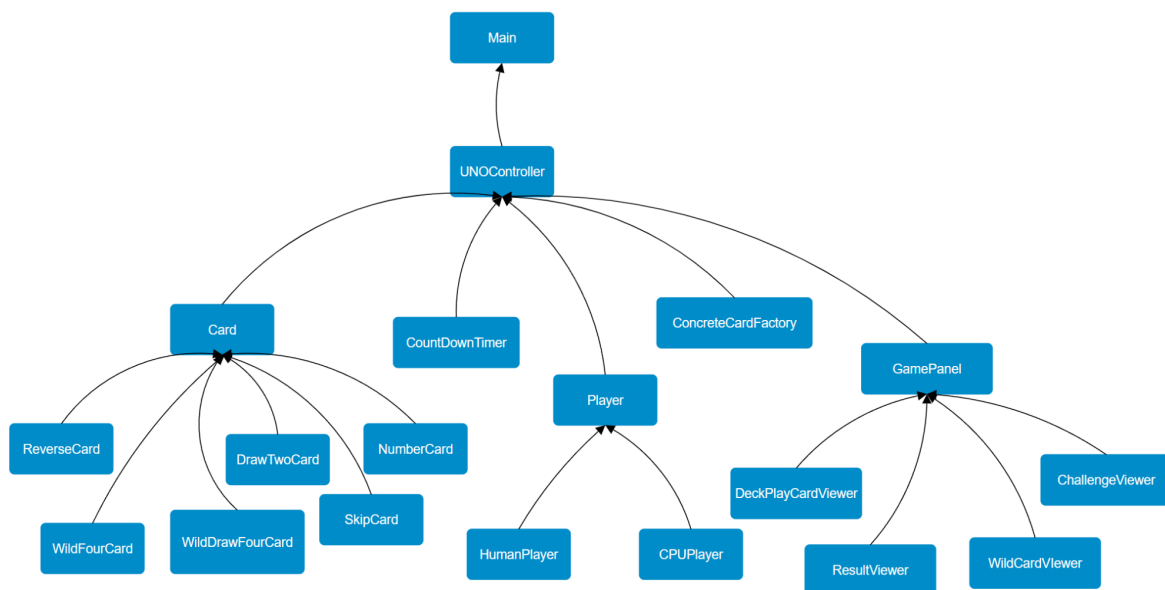
Conducted By:

Woo King Hei 58533231

Wong Hei Leong 58537010

1. Hierarchy Diagram	2
2. Testing Approach	2
3. Testing Process	2
4. Coverage Analysis	5
5. Code Refactoring	5
5.1 Renaming and Merging Methods/Variables	5
5.2 Extract Method	8
5.3 Substitute Algorithm	9
6. Test Cases	15
TestNumberCard.java Test Cases	15
TestSkipCard.java Test Cases	16
TestReverseCard.java Test Cases	16
TestDrawTwoCard.java Test Cases	17
TestWildCard.java Test Cases	17
TestWildDrawFourCard.java Test Cases	18
TestUNOController - Game Initialization Test Cases	18
TestUNOController - Player Management Test Cases	19
TestUNOController - Game Logic Test Cases	20
TestUNOController - Direction and State Test Cases	20
TestUNOController - Player Cards Test Cases	21
TestHumanPlayer.java Test Cases	21
TestCPUPlayer.java Test Cases	22
TestConcreteCardFactory.java Test Cases	22
TestUNOGamePanel.java Test Cases	23
TestWildCardViewer.java Test Cases	23

1. Hierarchy Diagram



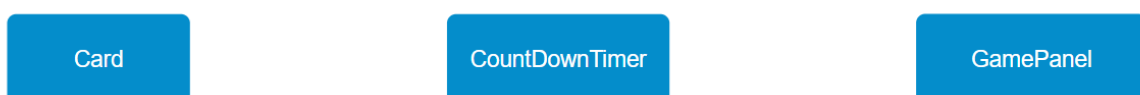
2. Testing Approach

We used a bottom-up approach for our testing process. We first grouped classes into core components such as Card, Player, and GamePanel. These are the fundamental parts of game operation which should be tested first. Hence, with the bottom up approach, we first conduct unit tests on these core components. Since these classes don't have any dependencies, they can be tested in isolation. Next we conducted integration testing by integrating related components such as card type classes (WildDrawFourCard, SkipCard, etc) and Card. Finally we conducted system testing to ensure the game can be run without errors.

We believe by using this approach, we can rely less on test stubs which improves testing efficiency. Also, since some classes are independent, different testers can test different components at the same time, saving time while not compromising coverage.

3. Testing Process

Our testing process is as follows:

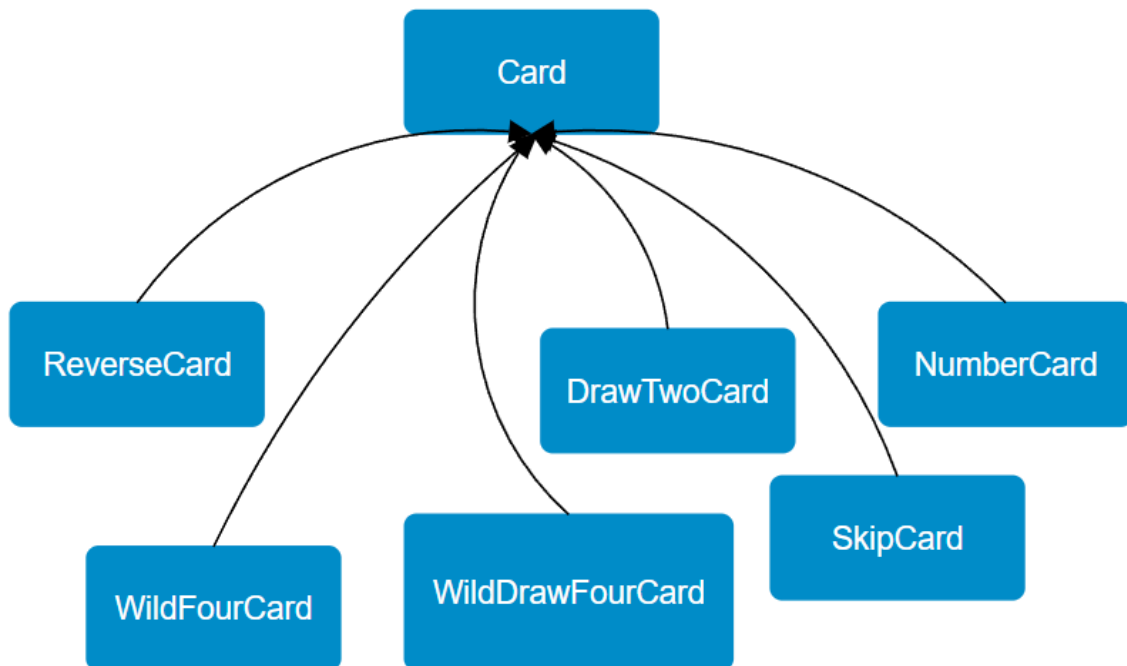


First, we conducted unit tests on core components:

- Card
- CountDownTimer
- ConcreteCardFactory.

These classes are able to be tested in isolation as they don't have any dependencies and work independently. For classes such as ReverseCard and HumanCard, although they are at the bottom of the hierarchy chain, they inherit functions from their respective superclasses, so they can only be tested during integration testing.

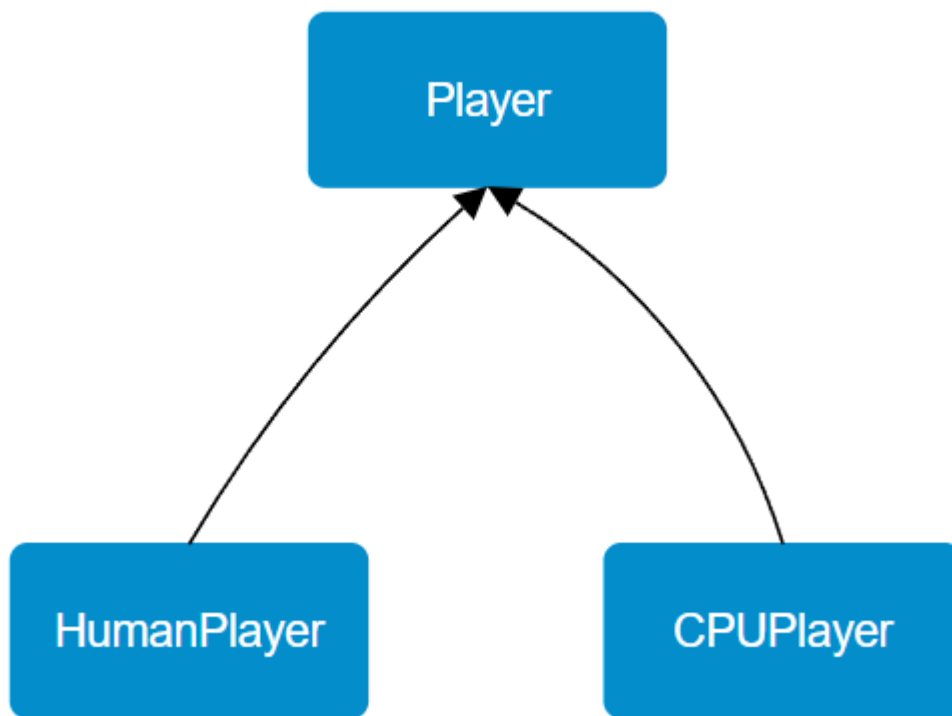
Next, we conduct integration testing, we broke into 3 parts that mostly share the same properties like value and partial methods:



Card components:

- Card + ReverseCard + WildFourCard + WildDrawFourCard + DrawTwoCard + SkipCard + NumberCard

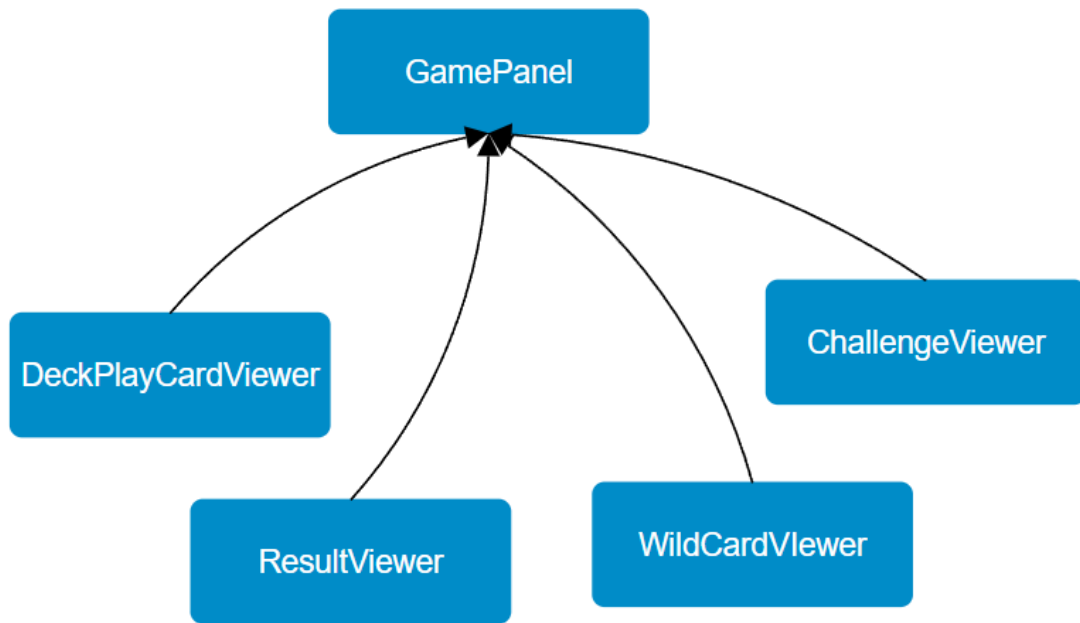
In these components, we mostly test on their unique methods for each type of card like canPlayCard(), cardFunction to ensure the core logic of the game is correct. Also, since the abstract class Card also shares the variable for the reference for the gamePanel to paint the card, we also test some of the variables like toString(), their x and y.



Player components:

- Player + HumanPlayer + CPUPlayer

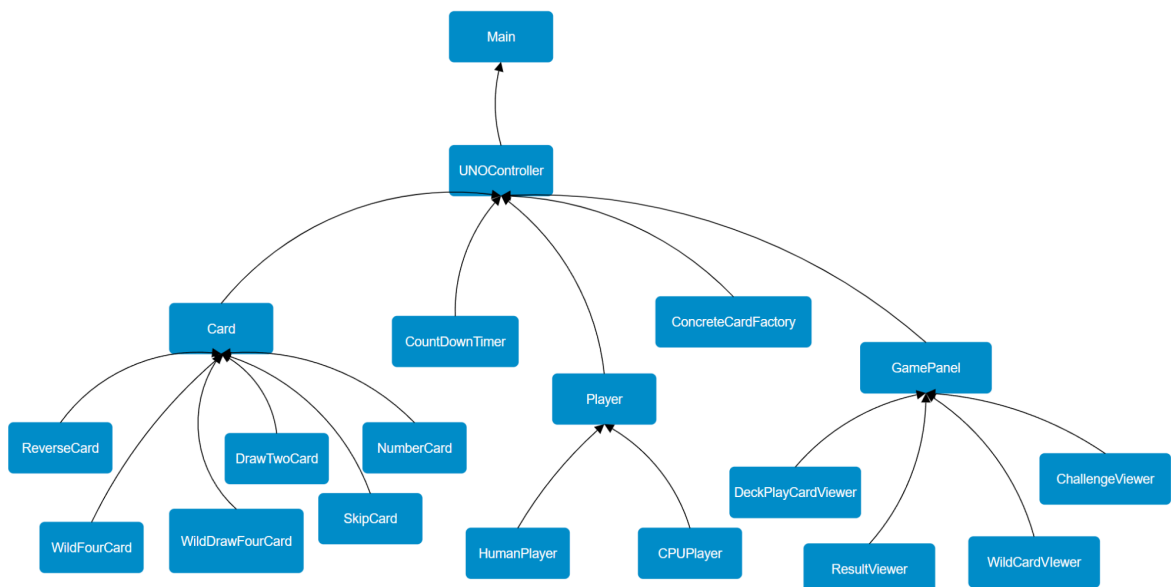
In these components, HumanPlayer mostly has to interact with the Viewer which is the User interface, so all the methods of actions will be different as CPUPlayer, so we have to test on each Player type's methods to ensure the method will not mix up. For example, CPUPlayer executed HumanPlayer's method accidentally which may call the viewer like "Challenge", "Choose to Play a Card" that may worsen players' experience.



GamePanel components:

- GamePanel + DeckPlayCardViewer + ResultViewer + WildCardViewer + ChallengeViewer

Every viewer components are very important since they are the way for players to interact with the core game logic. If there are any errors on any buttons, it may deceive players' experience also and some of the play-styles.



Lastly, system testing is completed, involving all classes, ensuring the game is run without any errors.

4. Coverage Analysis

5. Code Refactoring

Regarding the code refactoring aspect of our project, we frequently used 3 methods to improve code readability and modularization. The three methods include: Renaming and Merging Duplicate Code, Extract Method and Substitute Algorithm. Our code refactoring puts emphasis on model and controller components such as Cards and Players, as they contain the core game logic and state management that are performance-critical and platform-independent.

5.1 Renaming and Merging Methods/Variables

When using this method, our main objective is to improve variable or method names for clarity.

As we were developing the program, there were many occasions where we overlooked the variables' names and just used what was convenient at that time. As a result, the next time other team mates look back at the code, it is difficult for them to understand the code. Therefore, we changed some method names and variables' names to be more readable and deliver what they actually mean.

Moreover, we can mitigate code repetition and redundancy. In the following example, we initially put all the CPU Player's strategy and HumanPlayer's Strategy into one Player class. We then found out that there were many duplicated methods and variables such as the getter, setter name. Hence, we created an abstract class "Player", as well as dividing the player types into "HumanPlayer" and "CPUPlayer". These two concrete classes also implemented a Strategy Pattern, containing player type specific methods such as catchForgotUNO() and drawCard().

Before:

Player.java

```
public class Player {  
  
    protected String name;  
    protected ArrayList<Card> cards;  
    protected boolean isShout;  
    protected UNOController controller;  
    private Random random;  
    private boolean isHuman;  
  
}
```

Note: we initially used the boolean flag `isHuman` to identify whether the player object is human or CPU

```
    public String shoutUno() {  
        if (isHuman) {  
            String errorMessage = null;  
  
            if (this != controller.getCurrentPlayer() && this.getHand().size() > 1)  
{  
                errorMessage = "You have more than 1 card!";  
            }  
        }  
    }
```

```

        } else if (this != controller.getCurrentPlayer() &&
this.getHand().size() == 1) {
            isShout = true;
            System.out.println(name + " shouts UNO!");
        } else if (this.getIsShout()) {
            errorMessage = "You shouted UNO already!";
        } else if (this.getHand().size() > 2) {
            errorMessage = "You have more than 2 card!";
        } else {
            boolean hasPlayableCard = false;
            for (Card c : this.getHand()) {
                if (controller.canPlayCard(c, controller.getTopCard(1))) {
                    hasPlayableCard = true;
                }
            }
            if (!hasPlayableCard) {
                errorMessage = "You have no playable card!";
                return errorMessage;
            }
            isShout = true;
            System.out.println(name + " shouts UNO!");
        }
        return errorMessage;
    } else {
        if (!isShout && this.getHand().size() == 1) {
            isShout = true;
            System.out.println(name + " (CPU) shouts UNO!");
        } else {
            System.out.println(name + " (CPU) failed to shout UNO!");
        }
        return null;
    }
}

```

The above method ShoutUno() has code smell. The main reason for that is the “isHuman” flag is used to identify the player type (human or CPU) and decide which ShoutUno() logic to use.

After:

Player.java

```

public abstract class Player implements Drawable, Playable, Shoutable, Catchable,
Challengeable {
    protected String name;
    protected ArrayList<Card> hand;
    protected boolean isShout;
    protected UNOController controller;
}

```

CPUPlayer.java

```

public class CPUPlayer extends Player {

    private Random random;

    @Override
    public String shoutUno() {

```



```

        if (!isShout && this.getHand().size() == 1) {
            isShout = true;
            System.out.println(name + " (CPU) shouts UNO!");
        } else {
            System.out.println(name + " (CPU) failed to shout UNO!");
        }
        return null;
    }
}

```

HumanPlayer.java

```

public class HumanPlayer extends Player {

    private Random random;

    @Override
    public String shoutUno() {

        String errorMessage = null;
        if (this != controller.getCurrentPlayer() && this.getHand().size() > 1) {
            errorMessage = "You have more than 1 card!";
        } else if (this != controller.getCurrentPlayer() && this.getHand().size()
== 1) {
            isShout = true;
            System.out.println(name + " shouts UNO!");
        } else if (this.getIsShout()){
            errorMessage = "You shouted UNO already!";
        } else if (this.getHand().size() > 2) {
            errorMessage = "You have more than 2 card!";
        } else {
            boolean hasPlayableCard = false;
            for (Card c : this.getHand()){
                if(controller.canPlayCard(c, controller.getTopCard(1))){
                    hasPlayableCard = true;
                }
            }
            if (!hasPlayableCard) {
                errorMessage = "You have no playable card!";
                return errorMessage;
            }
            isShout = true;
            System.out.println(name + " shouts UNO!");
        }
        return errorMessage;
    }
}

```

5.2 Extract Method

This method finds a piece of code that does something specific, then moves it into a new method with a clear, descriptive name, leading to better readability and reusability. Moreover, this avoids high coupling and maintenance issues which makes classes loosely connected and easy to change.

In the following example, we encapsulated the logic which checks if a card can be played or not in the `canPlayCard()` method, this method is then stored within the class of each type of card. This approach avoids the controller operating directly on the value such as the color and number of cards to control the execution flow.

Before:

UNOController.java

```
public boolean canPlayCard(Card playedCard, Card matchCard) {
    // 如果没有顶牌（游戏刚开始或PlayedCard为空），允许任何牌
    if (matchCard == null) {
        return true;
    }

    switch (playedCard.getType()) {
        case Wild:
        case WildDrawFour:
            return true;
        case Skip:
        case Reverse:
        case DrawTwo:
            if (playedCard.getColor() == matchCard.getColor() ||
                playedCard.getType() == matchCard.getType()) {
                return true;
            }
            return false;
        case Number:
            if (playedCard.getColor() == matchCard.getColor()) {
                return true;
            } else if (matchCard.getType() == Type.Number && ((NumberCard)
                playedCard).getValue() == ((NumberCard) matchCard).getValue()) {
                return true;
            }
            return false;
        default:
            return false;
    }
}
```

As you can see, we directly call the getter for type and color of cards which may cause high coupling, not a good design.

After:

UNOController.java

```
public boolean canPlayCard(Card matchCard, Card topCard) {
    // 如果没有顶牌（游戏刚开始或PlayedCard为空），允许任何牌
    if (topCard == null || matchCard == null) {
        return true;
    }

    return topCard.checkCard(matchCard);
}
```

Card.java

```
abstract public boolean checkCard(Card matchCard);
```

Note: We added an abstract method since every Card Type class needs to use the method.

ReverseCard.java

```
@Override
public boolean checkCard(Card playedCard){
    if (playedCard.type == Type.Wild || playedCard.type == Type.WildDrawFour) {
        return true;
    }
    if (playedCard.color == this.color || playedCard.type == this.type) {
        return true;
    }
    return false;
}
```

Note: Here we use ReverseCard as an example of checking card type.

5.3 Substitute Algorithm

This method is implemented to replace an entire algorithm or method body with a different, clearer, or more efficient one. The main effects that we want to achieve with this method are to improve performance and allow the game to be played smoothly on computers that have lower specifications.

In this example, we originally used a simple ArrayList to store the whole deck which has a high time complexity and has relatively poor performance for different Card operations of the ArrayList. After refactoring the code, we changed it to a hashmap. By using a hashmap instead of a simple ArrayList, the time complexity of some operations can be drastically reduced. For instance, we optimized the time complexity of removing cards, searching cards and counting cards from $O(N)$ to $O(1)$.

Before:

UNOController.java

```
private void initializeGame() {
    Deck = new ArrayList<String>(Arrays.asList(
        "r0", "r1", "r2", "r3", "r4", "r5", "r6", "r7", "r8", "r9",
        "r1", "r2", "r3", "r4", "r5", "r6", "r7", "r8", "r9",
        "g0", "g1", "g2", "g3", "g4", "g5", "g6", "g7", "g8", "g9",
        "g1", "g2", "g3", "g4", "g5", "g6", "g7", "g8", "g9",
        "b0", "b1", "b2", "b3", "b4", "b5", "b6", "b7", "b8", "b9",
        "b1", "b2", "b3", "b4", "b5", "b6", "b7", "b8", "b9",
        "y0", "y1", "y2", "y3", "y4", "y5", "y6", "y7", "y8", "y9",
        "y1", "y2", "y3", "y4", "y5", "y6", "y7", "y8", "y9",
        "rSkip", "rReverse", "rDrawTwo",
        "rSkip", "rReverse", "rDrawTwo",
        "gSkip", "gReverse", "gDrawTwo",
        "gSkip", "gReverse", "gDrawTwo",
        "bSkip", "bReverse", "bDrawTwo",
        "bSkip", "bReverse", "bDrawTwo",
        "ySkip", "yReverse", "yDrawTwo",
        "ySkip", "yReverse", "yDrawTwo",
        "WildCard", "WildCard", "WildCard", "WildCard",
        "WildDrawFour", "WildDrawFour", "WildDrawFour", "WildDrawFour"));
}
```

```

        // Clear played cards
        PlayedCard.clear();
        // Clear each player's hand
        for (Player player : players) {
            player.getHand().clear();
        }
        // Stop turn timer
        turnTimer.stopTimer();
        turnTimer.startTimer(30);
        gamePanel.setIsGameEnd(false);
        // Reset other game state variables if needed
        isAction = false;

        PlayedCard.add(cardFactory.giveCard(Deck, true, true, "")); // 已打出的牌
顯示
        for (int i = 0; i < INITCARDSIZE; i++) {
            players.get(0).drawCard(cardFactory.giveCard(Deck, false, true, ""));
// 玩家的牌顯示
            for (int j = 1; j < 4; j++) {
                players.get(j).drawCard(cardFactory.giveCard(Deck, false, false,
"")); // CPU的牌隱藏
            }
        }
        currentPlayer = players.get(0);
        System.out.println("Current Player: " + currentPlayer.getName());
        playDirection = 1;
    }
}

```

ConcreteCardFactory.java

```

@Override
public Card createCard(ArrayList<String> Deck, Boolean isTop, boolean
isRevealed, String cardString) {
    int Decklen = Deck.size();
    if(isTop){
        Decklen = 56;
    }
    String card;
    if (cardString != "" && Deck.indexOf(cardString) != -1) { // create
specific card
        card = cardString;
        Deck.remove(Deck.indexOf(card));
    } else {
        if(cardString != ""){
            System.out.println("Deck doesn't contain this card!!!");
        }
        int index = random.nextInt(Decklen);
        card = Deck.get(index);
        Deck.remove(index);
    }

    if (card.equals("WildCard")){
        return new WildCard(isRevealed);
    }
}

```

```

    } else if (card.equals("WildDrawFour")){
        return new WildDrawFourCard(isRevealed);
    } else {
        Color cardcolor;
        switch (card.charAt(0)){
            case 'r':
                cardcolor = Color.Red;
                break;
            case 'b':
                cardcolor = Color.Blue;
                break;
            case 'y':
                cardcolor = Color.Yellow;
                break;
            default:
                cardcolor = Color.Green;
                break;
        }
        switch (card.substring(1)) {
            case "Skip":
                return new SkipCard(cardcolor, isRevealed);
            case "Reverse":
                return new ReverseCard(cardcolor, isRevealed);
            case "DrawTwo":
                return new DrawTwoCard(cardcolor, isRevealed);
            default:
                return new NumberCard(cardcolor,
Integer.parseInt(card.substring(1)), isRevealed);
        }
    }
}

```

Note: Apart from poorer performance, performing operations using ArrayList makes the code have code smell, which makes the code difficult to understand.

After:

UNOController.java

```

private void initializeGame() {
    Deck = new HashMap<>();

    for (String color : new String[] { "r", "g", "b", "y" }) {
        Deck.put(color + "0", 1);
        for (int i = 1; i <= 9; i++) {
            Deck.put(color + i, 2);
        }
    }

    for (String color : new String[] { "r", "g", "b", "y" }) {
        Deck.put(color + "Skip", 2);
        Deck.put(color + "Reverse", 2);
        Deck.put(color + "DrawTwo", 2);
    }

    Deck.put("WildCard", 4);
}

```

```

Deck.put("WildDrawFour", 4);

// Clear played cards
PlayedCard.clear();
// Clear each player's hand
for (Player player : players) {
    player.getHand().clear();
}
// Stop turn timer
turnTimer.stopTimer();
turnTimer.startTimer(30);
gamePanel.setIsGameEnd(false);
// Reset other game state variables if needed
isAction = false;

Card topCard = cardFactory.giveCard(Deck, true, true, "");
if (topCard == null) {
    deckEmpty();
    return;
}
PlayedCard.add(topCard); // 已打出的牌顯示

for (int i = 0; i < INITCARDSIZE; i++) {
    Card playerCard = cardFactory.giveCard(Deck, false, true, "");
    if (playerCard == null) {
        deckEmpty();
        return;
    }
    players.get(0).drawCard(playerCard); // 玩家的牌顯示
    for (int j = 1; j < 4; j++) {
        Card cpuCard = cardFactory.giveCard(Deck, false, false, "");
        if (cpuCard == null) {
            deckEmpty();
            return;
        }
        players.get(j).drawCard(cpuCard); // CPU的牌隱藏
    }
}
currentPlayer = players.get(0);
System.out.println("Current Player: " + currentPlayer.getName());
playDirection = 1;
}

```

ConcreteCardFactory.java

```

@Override
public Card createCard(HashMap<String, Integer> Deck, Boolean isTop, boolean
isRevealed, String cardString) {
    String card = null;

    if(Deck == null || Deck.isEmpty()) {
        return null;
    }
}

```

```

        if (isTop) {
            ArrayList<String> numberCards = getNumberCards(Deck);
            if (numberCards.isEmpty()) {
                return null;
            }
            card = getRandomCard(Deck, numberCards);
        } else if (!cardString.isEmpty() && Deck.containsKey(cardString) &&
Deck.get(cardString) > 0) { // create specific card
            card = cardString;
            removeCardFromDeck(Deck, card);
        } else {
            if (!cardString.isEmpty()) {
                System.out.println("Deck doesn't contain this card!!!");
            }
            ArrayList<String> availableCards = new ArrayList<>(Deck.keySet());
            if (availableCards.isEmpty()) {
                return null;
            }
            card = getRandomCard(Deck, availableCards);
        }

        if (card == null) {
            return null;
        }

        return createSpecificCard(card, isRevealed);
    }

    private static final Random random = new Random();

    private ArrayList<String> getNumberCards(HashMap<String, Integer> Deck) {
        ArrayList<String> numberCards = new ArrayList<>();
        for (String key : Deck.keySet()) {
            if (!key.equals("WildCard") && !key.equals("WildDrawFour") &&
                !key.contains("Skip") && !key.contains("Reverse") &&
!key.contains("DrawTwo")) {
                numberCards.add(key);
            }
        }
        return numberCards;
    }

    private void removeCardFromDeck(HashMap<String, Integer> Deck, String card) {
        Deck.put(card, Deck.get(card) - 1);
        if (Deck.get(card) == 0) {
            Deck.remove(card);
        }
    }

    private String getRandomCard(HashMap<String, Integer> Deck, ArrayList<String>
Cards) {
        int index = random.nextInt(Cards.size());

```

```

        String card = Cards.get(index);
        removeCardFromDeck(Deck, card);
        return card;
    }

    private Card createSpecificCard(String card, boolean isRevealed) {
        if (card.equals("WildCard")){
            return new WildCard(isRevealed);
        }
        if (card.equals("WildDrawFour")){
            return new WildDrawFourCard(isRevealed);
        }

        Color cardcolor = getColor(card);
        switch (card.substring(1)) {
            case "Skip":
                return new SkipCard(cardcolor, isRevealed);
            case "Reverse":
                return new ReverseCard(cardcolor, isRevealed);
            case "DrawTwo":
                return new DrawTwoCard(cardcolor, isRevealed);
            default:
                return new NumberCard(cardcolor,
Integer.parseInt(card.substring(1)), isRevealed);
        }
    }

    private Color getColor(String card) {
        switch (card.charAt(0)){
            case 'r':
                return Color.Red;
            case 'b':
                return Color.Blue;
            case 'y':
                return Color.Yellow;
            default:
                return Color.Green;
        }
    }
}

```

Note: we also used “Extract Method” to improve code readability and consistency in ConcreteCardFactory.java, the above is the reference of the called method.

6. Test Cases

TestNumberCard.java Test Cases

Test ID	Test Name	Input	Expected	Status
1	testNumberCard Creation	Create a NumberCard with color RED and value 5	Card created successfully with correct attributes	PASS

2	testGetColor	Get color of a NumberCard	Returns correct color	PASS
3	testGetValue	Get value of a NumberCard	Returns correct number value	PASS
4	testCanPlayOnSameColor	NumberCard on same color card	Returns true	PASS
5	testCanPlayOnSameNumber	NumberCard on same number card	Returns true	PASS
6	testCannotPlayOnDifferent	NumberCard on different color and number	Returns false	PASS
7	testCanPlayOnWild	NumberCard on Wild card	Returns true	PASS
8	testGetType	Get card type	Returns NUMBER	PASS
9	testToString	toString method	Returns correct string representation	PASS

TestSkipCard.java Test Cases

Test ID	Test Name	Input	Expected	Status
1	testSkipCardCreation	Create a SkipCard with color BLUE	Card created successfully	PASS
2	testGetColor	Get color of a SkipCard	Returns correct color	PASS
3	testCanPlayOnSameColor	SkipCard on same color card	Returns true	PASS
4	testCanPlayOnSkip	SkipCard on another SkipCard	Returns true	PASS
5	testCannotPlayOnDifferent	SkipCard on different color and type	Returns false	PASS
6	testGetType	Get card type	Returns SKIP	PASS
7	testToString	toString method	Returns correct string representation	PASS

TestReverseCard.java Test Cases

Test ID	Test Name	Input	Expected	Status
1	testReverseCardCreation	Create a ReverseCard with color GREEN	Card created successfully	PASS
2	testGetColor	Get color of a ReverseCard	Returns correct color	PASS
3	testCanPlayOnSameColor	ReverseCard on same color card	Returns true	PASS
4	testCanPlayOnReverse	ReverseCard on another ReverseCard	Returns true	PASS
5	testCannotPlayOnDifferent	ReverseCard on different color and type	Returns false	PASS
6	testCanPlayOnWild	ReverseCard on Wild card	Returns true	PASS
7	testGetType	Get card type	Returns REVERSE	PASS
8	testToString	toString method	Returns correct string representation	PASS

TestDrawTwoCard.java Test Cases

Test ID	Test Name	Input	Expected	Status
1	testDrawTwoCardCreation	Create a DrawTwoCard with color YELLOW	Card created successfully	PASS
2	testGetColor	Get color of a DrawTwoCard	Returns correct color	PASS
3	testCanPlayOnSameColor	DrawTwoCard on same color card	Returns true	PASS
4	testCanPlayOnDrawTwo	DrawTwoCard on another DrawTwoCard	Returns true	PASS

5	testCannotPlayOnDifferent	DrawTwoCard on different color and type	Returns false	PASS
6	testGetType	Get card type	Returns DRAW_TWO	PASS
7	testToString	toString method	Returns correct string representation	PASS

TestWildCard.java Test Cases

Test ID	Test Name	Input	Expected	Status
1	testWildCardCreation	Create a WildCard	Card created successfully	PASS
2	testCanPlayOnAnyCard	WildCard on any colored card	Returns true	PASS
3	testSetColor	Set chosen color to BLUE	Color set successfully	PASS
4	testGetChosenColor	Get chosen color	Returns correct chosen color	PASS
5	testGetColor	Get card color	Returns WILD	PASS
6	testGetType	Get card type	Returns WILD	PASS
7	testToString	toString method	Returns correct string representation	PASS

TestWildDrawFourCard.java Test Cases

Test ID	Test Name	Input	Expected	Status
1	testWildDrawFourCardCreation	Create a WildDrawFourCard	Card created successfully	PASS
2	testCanPlayOnAnyCard	WildDrawFourCard on any card	Returns true	PASS
3	testSetColor	Set chosen color to RED	Color set successfully	PASS
4	testGetChosenColor	Get chosen color		PASS

	lor			
5	testGetColor	Get card color	Returns WILD	PASS
6	testGetType	Get card type	Returns WILD_DRAW_F OUR	PASS
7	testGetDrawAmount	Get draw amount	Returns 4	PASS
8	testCanPlayWhenNoMatch	Valid wild draw four play	Returns true	PASS
9	testToString	toString method	Returns correct string representation	PASS

TestUNOController - Game Initialization Test Cases

Test ID	Test Name	Input	Expected	Status
1	testGameInitialization	Initialize UNO game with 4 players	Game initialized successfully	PASS
2	testDeckInitialized	Check deck initialization	Deck contains 108 cards	PASS
3	testPlayersDealtCards	Each player dealt initial cards	Each player has 7 cards	PASS
4	testDiscardPileInitialized	Check discard pile initialization	Has one card on top	PASS
5	testCurrentPlayerSet	Check current player initialized	Current player is first player	PASS
6	testDrawPileNotEmpty	Check draw pile has cards	Draw pile is not empty	PASS
7	testGameDirectionInit	Check initial game direction	Direction is CLOCKWISE	PASS
8	testGameNotOver	Check game is not over at start	Game is not over	PASS

TestUNOController - Player Management Test Cases

Test ID	Test Name	Input	Expected	Status
---------	-----------	-------	----------	--------

1	testGetCurrentPlayer	Get current player	Returns correct current player	PASS
2	testGetPlayers	Get all players	Returns list of all players	PASS
3	testNextPlayer	Move to next player	Current player changes correctly	PASS
4	testPlayerCount	Check number of players	Returns correct player count	PASS
5	testGetPlayerByIndex	Get player at specific index	Returns correct player	PASS
6	testAddPlayer	Add new player to game	Player added successfully	PASS
7	testCurrentPlayerIndex	Get current player index	Returns correct index	PASS

TestUNOController - Game Logic Test Cases

Test ID	Test Name	Input	Expected	Status
1	testPlayCard	Play valid card	Card played successfully	PASS
2	testDrawCard	Player draws a card	Card added to player hand	PASS
3	testCanPlayCard	Check if card can be played	Returns true for valid card	PASS
4	testGetTopCard	Get top card of discard pile	Returns correct top card	PASS
5	testCheckWinner	Check if player has won	Detects winner correctly	PASS
6	testHandleSkipCard	Play Skip card	Next player is skipped	PASS
7	testHandleDrawTwoCard	Play Draw Two card	Next player draws 2 cards	PASS
8	testHandleWildDrawFour	Play Wild Draw Four card	Next player draws 4 cards	PASS
9	testReshuffleDeck	Draw pile empty, reshuffle needed	Deck reshuffled successfully	PASS

TestUNOController - Direction and State Test Cases

Test ID	Test Name	Input	Expected	Status
1	testReverseDirection	Play Reverse card	Direction reversed	PASS
2	testGetDirection	Get current direction	Returns correct direction	PASS
3	testClockwiseDirection	Direction is clockwise	Next player calculated correctly	PASS
4	testCounterClockwiseDirection	Direction is counter-clockwise	Next player calculated correctly	PASS
5	testGameOver	Player has no cards	Game over detected	PASS
6	testIsGameOver	Check if game is over	Returns correct status	PASS
7	testGetWinner	Get winner of game	Returns correct winner	PASS
8	testResetGame	Reset game state	Game reset successfully	PASS
9	testHandleReverseCard	Handle Reverse card effect	Direction changes and turn proceeds	PASS

TestUNOController - Player Cards Test Cases

Test ID	Test Name	Input	Expected	Status
1	testGetPlayerCards	Get cards of a player	Returns player's card list	PASS
2	testPlayerHandSize	Check player hand size	Returns correct number of cards	PASS
3	testRemoveCardFromHand	Remove a card from player hand	Card removed successfully	PASS
4	testAddCardToHand	Add a card to player hand	Card added successfully	PASS

TestHumanPlayer.java Test Cases

Test ID	Test Name	Input	Expected	Status
1	testHumanPlayerCreation	Create human player with name	Player created successfully	PASS
2	testGetName	Get player name	Returns correct name	PASS
3	testAddCard	Add card to hand	Card added successfully	PASS
4	testRemoveCard	Remove card from hand	Card removed successfully	PASS
5	testGetHand	Get player's hand	Returns list of cards	PASS
	testGetHandSize	Get number of cards in hand	Returns correct size	PASS
7	testIsHuman	Check if player is human	Returns true	PASS

TestCPUPlayer.java Test Cases

Test ID	Test Name	Input	Expected	Status
1	testCPUPlayerCreation	Create CPU player with name	Player created successfully	PASS
2	testGetName	Get CPU player name	Returns correct name	PASS
3	testChooseCard	CPU chooses valid card	Returns playable card	PASS
4	testAddCard	Add card to CPU hand	Card added successfully	PASS
5	testGetHandSize	Get CPU hand size	Returns correct size	PASS
6	testIsHuman	Check if player is human	Returns false	PASS

TestConcreteCardFactory.java Test Cases

Test ID	Test Name	Input	Expected	Status
---------	-----------	-------	----------	--------

1	testCreateNumberCard	Create number card	Returns NumberCard instance	PASS
2	testCreateSkipCard	Create skip card	Returns SkipCard instance	PASS
3	testCreateReverseCard	Create reverse card	Returns ReverseCard instance	PASS
4	testCreateDrawTwoCard	Create draw two card	Returns DrawTwoCard instance	PASS
5	testCreateWildCard	Create wild card	Returns WildCard instance	PASS
6	testCreateWildDrawFourCard	Create wild draw four card	Returns WildDrawFourCard instance	PASS
7	testCardAttributes	Check created card attributes	Attributes match specifications	PASS
8	testFactoryReturnsCard	Factory returns Card interface	All cards implement Card	PASS
9	testMultipleCardCreation	Create multiple cards	All cards created independently	PASS
10	testCardTypesDistinct	Different card types	Each type is distinct	PASS

TestUNOGamePanel.java Test Cases

Test ID	Test Name	Input	Expected	Status
1	testPanelInitialization	Initialize game panel	Panel created successfully	PASS
2	testDisplayCards	Display player cards on panel	Cards displayed correctly	PASS
3	testUpdatePanel	Update panel with new game state	Panel updates correctly	PASS
4	testCardClickListener	Click on card	Card click detected	PASS
5	testDrawButtonAction	Click draw button	Draw action	PASS

	ction		triggered	
--	-------	--	-----------	--

TestWildCardViewer.java Test Cases

Test ID	Test Name	Input	Expected	Status
1	testViewerInitiali zation	Initialize wild card viewer	Viewer created successfully	PASS
2	testDisplayColor Options	Display color selection options	Four color buttons displayed	PASS
3	testRedButtonCli ck	Click red color button	Red color selected	PASS
4	testBlueButtonCli ck	Click blue color button	Blue color selected	PASS
5	testGreenButtonC lick	Click green color button	Green color selected	PASS
6	testYellowButton Click	Click yellow color button	Yellow color selected	PASS
7	testColorSelectio n	Select color and confirm	Color confirmed and returned	PASS
8	testDialogDisplay	Display color selection dialog	Dialog displays correctly	PASS
9	testCancelButton	Click cancel button	Dialog closes without selection	PASS
10	testGetSelectedC olor	Get selected color	Returns correct selected color	PASS

https://1drv.ms/p/c/6b78dadb63136a11/ET-AueBGvXZKkDR71a8NSGYBTysa_AUmxDxp15C8mPb6XA?e=ILCq7g